PREREQUISITE

GitHub Token Authentication for Terminal Pushes

Needed for every git push over HTTPS (Questions 2, 3 and 6)

When running `git push` over HTTPS on GitHub, account passwords are no longer accepted. Use a Personal Access Token (PAT) instead.

1. Navigate to **GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic)**.
2. Click **Generate new token (classic)**, give it a note (e.g. *Git Push Token*) and tick the **repo** scope.
3. Click **Generate token** and copy the token string immediately; GitHub will not show it again.
4. On `git push`, enter your GitHub **username**, then paste the **token** at the password prompt (nothing is echoed while pasting).

QUESTION 1

Jira Board and Sprint Management

Task Statement: Create a Jira board for a given project, create Product and Sprint Backlogs, add 5–6 user stories, start a sprint, move stories through To Do, In Progress, In Review, Done, and add appropriate comments.

Practical Execution Steps

1. **Create the project space:** Log in to Jira > **Create space** > select **Software development (Scrum)** > name it (e.g. *APSIT-Team-Sync*) > confirm the project key.
2. **Create an Epic:** click **+ Create** > Issue Type **Epic** > Summary: *DevOps Integration with Jira* > add a description > **Create**.
3. **Populate the Product Backlog (5–6 stories):** open **Backlog** > under the Backlog section click **+ Create work item** > type **Story**. Set the Epic as the parent of each story.
   - Story 1: Fork and clone repository in local machine (1 pt)
   - Story 2: Create feature branch from Jira issue key (2 pts)
   - Story 3: Configure GitHub Actions CI workflow file (3 pts)
   - Story 4: Write unit tests and verify local build (3 pts)
   - Story 5: Execute Docker build and container test run (5 pts)
   - Story 6: Configure webhook synchronization between GitHub and Jira (5 pts)
4. **Create and plan the Sprint:** click **Create sprint** in the Backlog view > name it (e.g. *SCRUM Sprint 1*) > set duration to 1 week > drag the stories from the Product Backlog into the sprint.
5. **Start the Sprint:** click **Start sprint** > enter the Sprint Goal (e.g. *Implement initial CI automation and GitHub integration*) > **Start**.
6. **Manage the active board:** open the **Board** (To Do, In Progress, In Review, Done). Drag a story to **In Progress** when development begins, to **In Review** when review/testing starts, and to **Done** once acceptance criteria are verified.
7. **Add comments (Action Map):** open a story > Activity/Comments > post an action map of milestones, e.g. `Fork done [x], Branch created [x], CI passing [x]` > **Save**.

Viva Questions & Answers

Q1: Define Product Backlog and Sprint Backlog. Differentiate between them.

Feature	Product Backlog	Sprint Backlog
Definition	A prioritized master list of all features, enhancements, bug fixes and technical requirements needed for the product.	The set of Product Backlog items selected for the sprint, plus the team's plan for delivering them.
Ownership	Owned and ordered by the Product Owner.	Owned and managed by the Developers (the Scrum Team members doing the work).
Timeframe	Long-term and continuous; evolves as needs change.	Short and time-boxed (usually 1–4 weeks).
Dynamism	Dynamic; items can be added, reordered or removed at any time.	Scope is protected during the sprint; the plan may be updated by Developers as they learn more, and scope changes are negotiated with the Product Owner.

Q2: How is the Sprint Process carried out in an Agile Team?

An Agile sprint follows a repeating five-phase cycle:

1. **Sprint Planning:** the Product Owner and team review the Product Backlog, select top-priority items based on team capacity/velocity, define the Sprint Goal and create the Sprint Backlog with story-point estimates.
2. **Sprint Execution:** developers and testers pull items into work and move tickets across the board (To Do → In Progress → In Review → Done).
3. **Daily Scrum (standup):** a 15-minute daily sync. Common format: what I did yesterday, what I will do today, any blockers.
4. **Sprint Review (demo):** at the end of the sprint the team demonstrates the working increment to stakeholders and collects feedback.
5. **Sprint Retrospective:** the team inspects what went well, what did not, and agrees improvements for the next sprint.

QUESTION 2

Jira and GitHub Integration

Task Statement: Create a Jira Story and add it to To Do. Clone the forked repository (https://github.com/shruthi1722/node_test.git), create a GitHub branch using the Jira story, and follow the GitHub Integration steps.

Practical Execution Steps

1. **Create the Story in Jira:** in your project create a Story titled *<YourName>: Integrate NPM Project with DevOps Integration;* keep status **To Do**; note the issue key (e.g. **SCRUM-7**).
2. **Fork the repo first:** open `github.com/shruthi1722/node_test` and click **Fork**, so you can push to your own copy.
3. **Install the GitHub integration:** in Jira go to **Apps > Explore more apps**, search **GitHub for Jira** (listed by Atlassian; on GitHub's side it appears as *GitHub for Atlassian*) > **Get it now**. Then **Apps > Manage apps > GitHub for Jira > Get started** > **GitHub Cloud** > authorize > select the `node_test` repository.
4. **Create the linked feature branch from the terminal:**

```
git clone https://github.com/<your-username>/node_test.git
cd node_test
git checkout -b SCRUM-7-integrate-npm-project
```

5. **Commit and push using the Jira issue key:** modify a file (e.g. `app.js` or `demo.txt`), then:

```
git status
git add .
git commit -m "SCRUM-7 Implement CI pipeline integration and test scripts"
git push -u origin SCRUM-7-integrate-npm-project
```

6. **Verify in Jira:** refresh the story. The **Development** panel should show **1 branch** and **1 commit** (and PR/build links if present).

Viva Questions & Answers

Q1: How can Jira issues be linked to GitHub branches and commits?

By including the unique Jira issue key (e.g. SCRUM-7) in development items:

- **Branch names:** `git checkout -b SCRUM-7-feature-auth`
- **Commit messages:** `git commit -m "SCRUM-7 Refactor database connection strings"`
- **Pull requests:** key in the PR title or description, e.g. *SCRUM-7: Add Node.js build configuration*

The GitHub–Jira integration picks up these keys when code is pushed and attaches the branch, commits, pull requests and build/deployment status to the issue's Development panel.

Q2: How is this useful for others to track changes?

- **End-to-end traceability:** anyone can open one ticket and see the branches, commits, PRs and builds for that requirement.
- **Less manual status reporting:** no need to ask 'is it coded?' or 'did tests pass?'; the ticket shows it.
- **Workflow automation:** Jira *smart commits* (e.g. `SCRUM-7 #in-review` in a commit message) and Jira automation rules can move issues between statuses, for example when a PR is opened or merged.
- **Auditability:** clear record of who changed what and when, which lowers release risk.

QUESTION 3

Git Commands and Version Control

Task Statement: Clone the forked repository (https://github.com/shruthi1722/node_test.git). Use GitHub commands to create a branch, list branches, create or modify a file, check status, add the file to the staging area, commit changes, and push to GitHub.

Practical Execution Steps & Command Log

```
# 1. Clone your personal fork (fork it on GitHub first)
git clone https://github.com/<your-username>/node_test.git
cd node_test

# 2. List local branches (current branch has an asterisk)
git branch

# 3. Create and switch to a new feature branch
git checkout -b test-branch          # modern equivalent: git switch -c test-branch

# 4. Verify you are on the new branch
git branch

# 5. Create a new file
echo "Hello from ASDD Practical Exam" > demo.txt

# 6. Check status (demo.txt shows RED under 'Untracked files')
git status

# 7. Stage the file (moves to GREEN, ready to commit)
git add demo.txt                     # or: git add .

# 8. Verify staged status
git status

# 9. Commit to the local repository
git commit -m "Add demo.txt for practical exam verification"

# 10. Push the branch to GitHub (username + token when prompted)
git push -u origin test-branch
```

Viva Questions & Answers

Q1: Explain git add, git commit, git push, git branch and git status.

Command	Area / Scope	Purpose
`git status`	Working directory & staging area	Shows the current branch, untracked files (red), modified files, and files staged for the next commit (green).
`git add`	Working directory → staging area	Moves changes into the staging area (index), marking them for the next snapshot.
`git commit`	Staging area → local repository	Records the staged snapshot in local history with a message, author, timestamp and unique commit hash.
`git branch`	Repository references	Lists local branches (asterisk = current); with a name it creates a branch, with `-d` it deletes one.
`git push`	Local repository → remote (GitHub)	Uploads local commits on a branch to the remote and updates the remote branch.

Q2: What is HEAD in Git?

HEAD is a pointer to your current position in the repository. Normally it points to the current **branch**, which in turn points to that branch's latest commit. When you switch branches, HEAD moves to the other branch. In a 'detached HEAD' state it points directly at a commit instead of a branch.

Q3: Explain GitHub Actions.

GitHub Actions is an event-driven CI/CD platform built into GitHub for automating build, test, lint, package and deploy tasks.

- **Workflows:** YAML files in `.github/workflows/`.
- **Triggers (`on:`):** events such as `push`, `pull_request` or scheduled cron.
- **Runners (`runs-on:`):** fresh virtual machines (e.g. `ubuntu-latest`) that execute the jobs.
- **Actions (`uses:`):** reusable building blocks, e.g. `actions/checkout@v4` to fetch the code and `actions/setup-node@v4` to install Node.js.

QUESTION 4

Docker Image Management and Container Execution

Task Statement: Pull a Docker image, list available Docker images, display running containers, run a container using the downloaded image in interactive mode, and verify container execution.

Practical Execution Steps & Command Log

```
# 1. Pull the official Node image from Docker Hub
sudo docker pull node

# 2. List locally downloaded images
sudo docker images

# 3. Display running containers (likely empty at this point)
sudo docker ps

# 4. Run a container interactively and start Bash
sudo docker run -it node bash

#    --- inside the container (root@<container-id>:/#) ---
node --version
npm --version
ls

#    (Optional) open a SECOND terminal now and run: sudo docker ps
#    to show the container running.  Then return and type:
exit                        # leaves the shell and stops the container

# 5. List running containers (the exited one will NOT appear)
sudo docker ps

# 6. List ALL containers, including stopped/exited ones
sudo docker ps -a

# 7. Optional clean-up
# sudo docker rm <container_id>
# sudo docker rmi node
```

Viva Questions & Answers

Q1: Differentiate between a Docker Image and a Docker Container.

Feature	Docker Image	Docker Container
Definition	A read-only, immutable package with application code, runtime, tools, libraries and configuration.	A runnable, isolated instance created from an image (with a thin writable layer on top).
State	Static; a stored template on disk.	Active; a running (or stopped) process isolated using Linux namespaces and cgroups.
Analogy	A class in OOP, or a recipe.	An object instance, or the baked cake.
Multiplicity	Built once from a Dockerfile or pulled from Docker Hub.	Many independent containers can be started from one image.

Q2: Explain the Docker build command.

`docker build` reads the instructions in a Dockerfile and packages the application environment into an image.

```
docker build -t <image-name>:<tag> <context-path>
docker build -t my-node-app:v1 .
```

- `-t my-node-app:v1` tags the image with a name and version.
- `.` is the build context: the directory sent to Docker, containing the Dockerfile, source and `package.json`.
- Instructions run in order and each produces a cached layer, so unchanged layers are reused on rebuilds.

QUESTION 5

Dockerfile Creation and Image Building

Task Statement: Create a Dockerfile for a given application, build a Docker image, run a container using the image with port forwarding, and verify that the application is working.

Practical Execution Steps

1. **Prepare the project:** be in `node_test`, which contains `package.json` and `app.js`.
2. **Create the Dockerfile** (`nano Dockerfile`):

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

3. **(Best practice) Create** `.dockerignore`**:**

```
node_modules
.git
npm-debug.log
```

4. **Build the image:**

```
sudo docker build -t my-node-app .
```

5. **Verify the image:** `sudo docker images`
6. **Run with port mapping:**

```
sudo docker run -d --name my-running-app -p 3000:3000 my-node-app
```

7. **Verify:** run `sudo docker ps`, then `curl http://localhost:3000` or open `http://localhost:3000` in a browser.

Viva Questions & Answers

Q1: What is a Dockerfile?

A plain-text file of sequential instructions that Docker executes to assemble an image automatically and repeatably.

Q2: Explain FROM, WORKDIR, COPY, RUN, EXPOSE and CMD.

Instruction	When it acts	Purpose
`FROM`	Build time	Sets the base image (e.g. `node:22`), giving the OS and runtime layer. It starts the image (only ARG may precede it).
`WORKDIR`	Build and run time	Sets the working directory inside the image (e.g. `/app`); later COPY, RUN and CMD use it.
`COPY`	Build time	Copies files from the build context into the image (e.g. `COPY package*.json ./`).
`RUN`	Build time	Runs shell commands while building (e.g. `RUN npm install`); each RUN creates a layer.
`EXPOSE`	Metadata / documentation	Documents the port the app listens on. It does not publish the port; use `-p` at run time.

Instruction	When it acts	Purpose
`CMD`	Container start	Default command when the container starts (e.g. `["node","app.js"]`); can be overridden on the command line.

QUESTION 6

Terraform and GitHub Actions

Task Statement: Fork the Terraform project (https://github.com/shruthi1722/terraform23), clone it to the local machine, modify main.tf, commit and push changes, and verify GitHub Actions workflow execution.

Practical Execution Steps

1. **Fork and clone:**

```
git clone https://github.com/<your-username>/terraform23.git
cd terraform23
```

2. **Modify** `main.tf` (`nano main.tf`):

```
provider "local" {}

resource "local_file" "demo" {
  filename = "hello.txt"
  content  = "Hello from Terraform via GitHub Actions!\nImplemented by <Your Name>."
}
```

3. **Workflow file** `.github/workflows/terraform.yml`:

```
name: Terraform Local

on:
  push:
    branches:
      - main

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan

      - name: Terraform Apply
        run: terraform apply -auto-approve

      - name: Verify Created File
        run: cat hello.txt
```

4. **Commit and push:**

```
git add main.tf
git commit -m "Update main.tf for Terraform automation testing"
git push origin main
```

5. **Verify:** GitHub repo > **Actions** tab > open the *Terraform Local* run > confirm the green tick > open **Verify Created File** and check that the `hello.txt` content is printed.

Viva Questions & Answers

Q1: What is Terraform and what is Infrastructure as Code (IaC)?

Terraform is an open-source IaC tool from HashiCorp. Infrastructure is described in a declarative language, HashiCorp Configuration Language (HCL), and Terraform provisions it on cloud or on-premise platforms.

Infrastructure as Code means managing servers, networks, storage and load balancers through version-controlled configuration files instead of manual clicks in a cloud console, giving repeatability, review and rollback.

Q2: List the core Terraform commands and their purpose.

Command	Purpose
`terraform init`	Initializes the working directory: downloads provider plugins (AWS, Azure, local, etc.) and sets up the backend.
`terraform plan`	Shows an execution plan without changing anything: compares the desired state in the .tf files with current state and lists what will be created, changed or destroyed.
`terraform apply`	Carries out the planned changes and updates the state file. `-auto-approve` skips the confirmation prompt (used in CI/CD).
`terraform destroy`	Removes all resources managed by the configuration, avoiding unnecessary cost.

Q3: What is the purpose of the Terraform state file (terraform.tfstate)?

It maps the resources declared in your configuration to the real resources that were created, stores their metadata, and lets Terraform work out what must change on the next plan, including detecting drift when something is changed outside Terraform.

QUESTION 7

Grafana Using Docker

Task Statement: Pull and run the Grafana Docker image, map the container port to localhost:3001, and verify that the Grafana dashboard is accessible in a web browser.

Practical Execution Steps

1. **Pull and run the Grafana container:**

```
sudo docker pull grafana/grafana
sudo docker run -d --name grafana -p 3001:3000 grafana/grafana
```

`-p 3001:3000` maps host port 3001 to port 3000 inside the container (Grafana's default).

2. **Verify the container:** `sudo docker ps`. It should show image `grafana/grafana`, status **Up** and ports `0.0.0.0:3001->3000/tcp`.
3. **Open Grafana:** browse to `http://localhost:3001`. Default login: **admin / admin**. Set a new password or click **Skip**.
4. **Add a test data source and build a dashboard:**
   - **Connections > Add new connection**, search **TestData**, choose it and click **Add new data source**.
   - Click **Save & test** (expect 'Data source is working').
   - **Dashboards > New dashboard > Add visualization**, select the TestData data source (`grafana-testdata-datasource`), scenario **Random Walk**, view the graph and click **Save dashboard**.

Viva Questions & Answers

Q1: What is Grafana, and how is it used for monitoring and visualization in DevOps?

Grafana is an open-source analytics and visualization platform. It connects to data sources such as Prometheus, InfluxDB, Elasticsearch, AWS CloudWatch and MySQL to query, visualize and alert on metrics. In DevOps it provides:

- **Unified dashboards:** CPU, memory, disk and network metrics and application data (latency, 5xx error rate) on one screen.
- **Alerting:** notifications to Slack, PagerDuty or email when a metric crosses a threshold (e.g. CPU above 85% for 5 minutes).
- **Faster incident resolution:** helps engineers find bottlenecks, diagnose root causes and watch recovery after a deployment.

Q2: How do Prometheus and Grafana work together in a DevOps pipeline?

- **Prometheus** is the metric collector and time-series database. It periodically scrapes metrics from servers and containers and stores them.
- **Grafana** is the visualization layer. It queries Prometheus with PromQL and shows the data as graphs, gauges, bar charts and heat maps.